

CosmosX Mercury

Blockchain Foundations redesigned as a discovery-first mission campaign

CORE PROMISE

Players repair a spacecraft communication system. They experience why blockchain exists before receiving an explanation. The aim is not to cover the most terms; it is to make the essential ideas stick.

What this document contains

This is the complete content blueprint for Mercury. It contains the campaign story, lesson loop, exact mission copy, player actions, discovery moments, short theory unlocks, failure feedback, debriefs, and rewards for all eight modules. It is deliberately written without site implementation details.

Mercury learning outcome

By the end of Mercury, a complete beginner can explain why blockchain was invented, how transactions become blocks, how hashing and chained references expose tampering, how distributed nodes and consensus create a shared history, and when blockchain is or is not the right architecture.

Non-negotiable teaching rules

- Curiosity before vocabulary. The player sees or causes the problem before the term appears.
- Action before explanation. Every concept starts with a manipulation, decision, simulation, or repair.
- Short theory after discovery. Each theory unlock is one compact explanation, not a lecture.
- Productive failure. Wrong moves reveal a system consequence and a clear recovery path.
- Real-world intuition first. Use familiar ideas such as bank statements, Google Drive, UPI, delivery tracking, and Git history before blockchain jargon.
- No blockchain evangelism. Mercury teaches the tradeoffs and gives players permission to choose a database.
- Mercury stays blockchain-agnostic. Stellar is only a brief optional federated-consensus preview; deep Stellar development begins on later planets.

The player loop

Beat	Purpose
1. Mission briefing	One urgent story beat. The player knows what is at risk, not the answer.
2. Observe	A system visibly fails, conflicts, or behaves unexpectedly.
3. Interact	The player clicks, drags, routes, compares, deploys, or decides.
4. Discover	The interface names the pattern the player has just noticed.
5. Theory unlock	A 30 to 60 second explanation gives the pattern its real name and connects it to life outside the game.
6. Replay twist	A short variation checks transfer of understanding without becoming a quiz.
7. Reflection and reward	The player explains the consequence in plain language and earns XP, a rocket part, and planet progress.

Campaign arc

Mercury begins with one failed central database and ends with a launch architecture decision. The player does not merely learn a list of blockchain parts. They rebuild the exact capabilities that a single system lacks: an auditable ledger, blocks, tamper-evident fingerprints, linked history, resilient peer communication, network agreement, and sound judgment about when the whole stack is warranted.

Module	Player fantasy	Rocket part
01	Survive a central blackout	Launch Platform
02	Become an emergency ledger clerk	Fuel Tank
03	Pack the chain's cargo	Combustion Chamber
04	Catch invisible tampering	Rocket Engine
05	Stop a historical chain reaction	Structural Frame
06	Build a storm-proof mesh	Communications Array
07	Agree without a captain	Navigation System
08	Choose the right system before launch	Heat Shield

Why Does Blockchain Exist?

Mercury Central Command is the only place that knows who owns the colony's fuel. When it fails, every station becomes blind. The player feels the convenience of a central system before discovering its dependency on trust.

ROCKET PART UNLOCKED

Launch Platform - the rocket can stand, but it cannot leave until the record system is trustworthy.

Player leaves able to

- Explain why a trusted central database is often fast, useful, and appropriate.
- Identify a single point of failure and the risks of silent record changes.
- Explain why independent parties may need a shared record without one owner.
- Decide when a normal database is better than a blockchain.

Mission briefing

ON-SCREEN TRANSMISSION

Emergency: a solar flare has taken Mercury Central Command offline. Five stations cannot see their fuel balances, and one disputed transfer may have been edited. Bring the launchpad back online without pretending the problem never happened.

Mission beat 1: The black screen

CENTRALIZED SYSTEM / SINGLE POINT OF FAILURE

Observe

A simple station dashboard works beautifully: five stations request fuel, one central command approves it, and each balance updates instantly. The storm hits. The command node goes red; every station dashboard changes to 'Awaiting authority'.

Player does

The player taps each station. None can confirm a balance, release fuel, or settle a dispute. They can restart a client, but cannot restore the missing source of truth. They choose which launch function is blocked first: payment, audit, or emergency release.

The system responds

Every choice is blocked for the same reason: all decisions point back to Central Command. A mission overlay draws five arrows into one server and labels it 'one owner, one decision point'.

Discovery

A central database is not bad; it is efficient when its owner is trusted and available. The weakness is that all participants inherit its failure or its decisions.

Theory unlock

ONLY AFTER THE ACTION

Think of a bank, Google Drive, or a game server: one operator can make the system fast and simple. But if that operator is offline, hacked, dishonest, or compelled to change a record, everyone else is stuck. This is a single point of failure.

Debrief and reward

You did not fail to find a backup button. There was no independent record to consult. +20 XP: Failure Map recovered.

Productive failure

IF THE PLAYER GETS IT WRONG

If the player keeps retrying a station, the companion says: 'You are asking a mirror to answer a question only the missing server knows.'

Mission beat 2: The silent edit

TRUST PROBLEM / AUDITABILITY

Observe

Command returns briefly. A station claims it sent 80 fuel cells; Central Command now shows 30. Both screens look polished and official.

Player does

The player uses a rewind scrubber to compare the station's receipt, the command record, and a maintenance officer's export. They must place the three records on a timeline and flag the moment where one value changes without a matching event.

The system responds

The station receipt and export agree. Only the central record changed. The interface cannot prove who changed it; it can only show that the owner had the power to do so.

Discovery

When one party owns the record, everyone else must trust it to be honest, available, and careful. A dispute becomes 'whose database wins?' rather than 'what do all of us independently verify?'.

Theory unlock

ONLY AFTER THE ACTION

A shared database can be excellent when the administrator is accepted by everyone. Blockchain was invented for a narrower problem: multiple parties need to write to and verify a shared history, but no single party should be able to rewrite it alone.

Debrief and reward

You found the problem before anyone named it: trust was concentrated in one operator. +20 XP: Audit Lens installed.

Mission beat 3: Choose the repair architecture

DATABASE VERSUS BLOCKCHAIN

Observe

Three repair tickets arrive: a private cafeteria inventory app, a shared fuel-settlement log for rival stations, and a public announcement board.

Player does

The player drags each ticket onto one of four bays: 'normal database', 'shared database', 'blockchain', or 'no persistent system'. They then see a short simulation of cost, speed, and trust consequences.

The system responds

The cafeteria runs best on a normal database. The announcement board needs no shared ownership record. The rival-station settlement log benefits from independent verification because no station is accepted as the permanent owner.

Discovery

Blockchain is not a faster database. It trades speed, simplicity, and resource cost for shared verification where writers do not fully trust one another.

Theory unlock

ONLY AFTER THE ACTION

Use a blockchain when several parties must maintain important shared state, do not fully trust one another, and cannot or will not appoint a trusted middleman. If one trusted organization can safely operate the system, a normal database is usually better.

Debrief and reward

You chose tools for the problem instead of forcing every problem onto a blockchain. Launch Platform secured. +30 XP.

MODULE 02 | FIRST LIGHT

Transactions and Digital Ledgers

The player becomes Mercury's emergency ledger clerk. Instead of memorizing fields, they move oxygen and fuel through a living record, then catch the transaction that should never have been allowed through.

ROCKET PART UNLOCKED

Fuel Tank - verified transfers can now supply the rocket.

Player leaves able to

- Recognize a transaction as a request to change a shared record.
- Read sender, receiver, amount or data, fee, signature, and transaction ID as useful evidence.
- Explain an append-only ledger and calculate a balance from history.
- Trace a transaction from creation through broadcast, mempool, validation, and block inclusion.
- Reject invalid or double-spend attempts using balance, signature, and sequence checks.

Mission briefing

ON-SCREEN TRANSMISSION

Oxygen is low. Transfer requests are arriving faster than the crew can inspect them. Keep the ledger truthful long enough to fill the rocket's fuel tank.

Mission beat 1: Move the oxygen

TRANSACTION ANATOMY / APPEND-ONLY LEDGER**Observe**

Station A has 100 oxygen units. Station B has 20. A crew member sends 35 units from A to B. A live ledger line appears; neither old balance is erased.

Player does

The player assembles a transfer card by dragging a sender badge, receiver badge, amount dial, fee chip, signature seal, and receipt ID into the dispatch tray. They send two transfers, then use the ledger timeline to reconstruct each station's current balance.

The system responds

The balance display is derived from the entries: incoming values add, outgoing values subtract. When the player opens an old line, it is still present; a correction arrives as a new line rather than an overwrite.

Discovery

A transaction is an instruction that changes state. A ledger keeps the story of those changes, so anyone can audit how a balance became what it is.

Theory unlock

ONLY AFTER THE ACTION

A blockchain transaction is a signed message requesting a state change: transfer value, create an asset, or record data. An append-only ledger does not edit history in place. It keeps a chronological trail, much like a bank statement or Git commit history.

Debrief and reward

You did not type in a new balance. You created the events that explain it. +20 XP: Ledger Reader acquired.

Mission beat 2: The waiting room

TRANSACTION LIFECYCLE / MEMPOOL

Observe

Six transfer drones launch at once. They do not appear in the permanent ledger immediately. Instead, they collect in a glowing queue outside the validation gate.

Player does

The player drags lifecycle tiles into order: create, sign, broadcast, wait in the mempool, validate, include in a block. They can pause the network at any tile and inspect what a transaction is waiting for.

The system responds

A signed but unbroadcast transfer changes nothing. A broadcast transfer can wait in the mempool. Only a validated transfer placed in an accepted block becomes part of the shared history.

Discovery

Clicking Send is not the same as being recorded. A network needs a place for pending requests and a process for deciding which are valid.

Theory unlock

ONLY AFTER THE ACTION

The mempool is a waiting area for transactions that have been shared with the network but are not yet confirmed. Validators examine them before they become part of a block. Different networks expose this process differently, but the core idea is the same: submitted is not final.

Debrief and reward

You watched a transaction earn its place in history. +20 XP: Queue Stabilizer installed.

Mission beat 3: Gatekeeper under pressure

VALIDATION / DOUBLE SPEND

Observe

A station with 40 fuel units tries to send 30 units to two different stations at almost the same time. Another request has a broken signature; a third repeats an old sequence number.

Player does

The player works a fast visual gate. For each request they pull three levers: balance, signature, and sequence. Valid requests pass into the queue; invalid requests bounce back with a visible reason. The player must accept the first valid 30-unit transfer and reject the conflicting second one.

The system responds

After the first transfer reserves the available balance, the second can no longer be safely accepted. The queue displays its conflict rather than silently paying both recipients.

Discovery

Validation does not merely check whether a message looks tidy. It protects the shared record from funds being spent twice, forged requests, and replayed instructions.

Theory unlock

ONLY AFTER THE ACTION

Nodes check rules before accepting a transaction: enough balance for amount plus fee, a valid authorization signature, and the next valid sequence or nonce. Double spending is the attempt to use the same value twice; ordering and validation let the network accept one legitimate path and reject the conflict.

Debrief and reward

The oxygen queue stayed truthful because you rejected evidence, not people. Fuel Tank filled. +30 XP.

MODULE 03 | SOLAR RISE

Blocks and Blockchain Structure

The player packs validated transactions into cargo containers. Capacity is limited, order matters, and the first container establishes the world the rest of the chain must inherit.

ROCKET PART UNLOCKED

Combustion Chamber - transactions now travel as verified cargo.

Player leaves able to

- Explain why transactions are bundled into blocks.
- Identify a block's header metadata and body of transactions.
- Recognize the genesis block and its all-zero previous reference.
- Describe throughput as transactions per block divided by block time.
- Place blocks in chronological order using height and previous references.

Mission briefing

ON-SCREEN TRANSMISSION

The ledger queue is overflowing. The rocket needs orderly cargo containers, not a blizzard of individual requests. Build the first stable chain of launch records.

Mission beat 1: Pack the cargo block

BLOCK BODY / HEADER**Observe**

Twelve validated transfers float in a queue. A cargo container has room for four. Its front panel remains blank while the transaction chamber fills.

Player does

The player packs four transfers, then turns the container around to complete its header: height, timestamp, previous-block reference, transaction root, and a protocol value. They may inspect only the header of a distant container to quickly identify its place in the chain.

The system responds

The body answers 'what happened here?'. The header answers 'where does this block belong and how can it be checked?'. The full cargo list is heavier to inspect than the label.

Discovery

Blocks batch work. Headers carry compact metadata that lets a network organize and check the batch without rereading every transaction first.

Theory unlock

ONLY AFTER THE ACTION

A block is a container of validated transactions plus a header. The header commonly includes a height, timestamp, previous block hash, and fields required by the network's rules. Networks differ, but separating a compact header from the transaction body makes history easier to navigate and verify.

Debrief and reward

You turned many requests into one accountable unit. +20 XP: Cargo Cradle calibrated.

Mission beat 2: The first page

GENESIS BLOCK / INITIAL STATE

Observe

A sealed 'Block 0' container arrives with no parent cable. Its previous reference is a row of zeros. Inside are Mercury's initial fuel allocations and network settings.

Player does

The player opens three candidate genesis manifests and finds the one with a height of 0, all-zero previous reference, initial allocations, and the network identifier. They install it before any later container can lock into place.

The system responds

When the player tries to give Block 0 a parent, the docking clamps reject it: nothing existed before the first block on this chain.

Discovery

Every chain needs a known starting state. It is not a mysterious exception; it is the first agreed page of the ledger.

Theory unlock

ONLY AFTER THE ACTION

The genesis block is the first block in a blockchain. It has no previous block, so its previous reference is conventionally an all-zero value or another network-defined placeholder. It establishes the initial state from which later records grow.

Debrief and reward

You found the beginning instead of treating history as if it appeared from nowhere. +20 XP: Genesis Lock engaged.

Mission beat 3: Capacity versus speed

THROUGHPUT / CHRONOLOGICAL CHAIN

Observe

Mission Control offers two conveyor settings: 20 transactions every 10 seconds, or 100 transactions every 60 seconds. A scrambled run of four containers waits nearby.

Player does

The player uses a physical rate dial to compare transactions per second, then reconnects the scrambled containers by matching each height and previous reference. A bottleneck gauge shows that larger or more frequent blocks carry tradeoffs in bandwidth, storage, and verification work.

The system responds

Both conveyor settings yield different throughput. The chain only lights when the blocks form a strict chronological path: Block 0 to Block 1 to Block 2 to Block 3.

Discovery

Capacity is a design tradeoff, not a free upgrade. A blockchain is a timeline of batches, not a random pile of records.

Theory unlock

ONLY AFTER THE ACTION

A simple throughput estimate is transactions per block divided by block time. Raising capacity can improve throughput but can also make blocks more expensive to send, store, and verify. Each block points back to its predecessor, creating an ordered history.

Debrief and reward

You optimized for a real system constraint and rebuilt the timeline. Combustion Chamber unlocked. +30 XP.

Hashing and Data Integrity

The player learns the safety signal of a blockchain by breaking it. One changed character turns a familiar transmission into a completely unfamiliar fingerprint.

ROCKET PART UNLOCKED

Rocket Engine - every critical launch file now carries a tamper-evident seal.

Player leaves able to

- Describe a cryptographic hash as a deterministic, fixed-length fingerprint of data.
- Experience the avalanche effect rather than memorizing it.
- Use hashes to detect altered data.
- Distinguish hashing for integrity from encryption for confidentiality.
- Recognize that a hash is not a password, an encryption key, or proof that content is good.

Mission briefing

ON-SCREEN TRANSMISSION

An engine-control message was changed somewhere between the launch bay and the cockpit. You cannot trust its appearance. Build a way to notice even invisible tampering.

Mission beat 1: One character, total failure

HASH PROPERTIES / AVALANCHE EFFECT

Observe

The player verifies the command 'OPEN BAY 3' against a known engine seal. It matches. They change only the 3 to a 4; the seal shatters into an entirely different pattern.

Player does

The player uses a split console to edit messages letter by letter and compare their fingerprints. They test the same message twice, a short message, a long message, and a one-character change.

The system responds

Identical input always gives the same output. Different-sized messages produce outputs of the same visual length. A tiny edit produces a radically different output.

Discovery

A hash behaves like a digital fingerprint: repeatable for the same data, compact, and extremely sensitive to changes. It does not preserve a readable resemblance to the original.

Theory unlock

ONLY AFTER THE ACTION

A cryptographic hash maps data of any size to a fixed-size output. The same input gives the same hash; changing even a tiny part should radically change it. Modern hash functions are designed so you cannot feasibly reverse a hash to recover the original input or deliberately find two useful inputs with the same hash.

Debrief and reward

You changed one character and watched the whole fingerprint change. +20 XP: Integrity Scanner online.

Mission beat 2: Seal or secret?

HASHING VERSUS ENCRYPTION

Observe

Two tools sit in the launch lab. The first turns a maintenance log into a one-way checksum. The second locks a private message and later opens it with a key.

Player does

The player routes six mission needs to the right tool: verify a downloaded file, hide a private route, store a password safely, recover a confidential briefing, compare two cargo manifests, and send a secret message. They can test each choice with a 'Can I recover the original?' switch.

The system responds

Hashing returns a repeatable fingerprint but never recreates the original message. Encryption turns readable data into protected ciphertext and requires the correct key to recover it.

Discovery

Both are cryptographic tools, but they solve different problems. One checks whether data changed; the other keeps data private.

Theory unlock

ONLY AFTER THE ACTION

Hashing is primarily for integrity and verification. Encryption is for confidentiality: someone with the correct key can decrypt the protected content. A hash is not encrypted data, and encryption alone does not make a public ledger tamper-evident.

Debrief and reward

You stopped using one word, 'secure', for two different jobs. +20 XP: Crypto Toolbelt assembled.

Mission beat 3: Find the altered transmission

CHECKSUM INTEGRITY AUDIT

Observe

Four launch transmissions look nearly identical. Mission Control publishes the expected fingerprints before the files arrive. One message contains a single altered word.

Player does

The player drags each incoming file to the verifier. They compare calculated and expected seals, quarantine the mismatch, then inspect it to find the tiny human-readable edit. A second round asks the player to match three configuration commands to their known seals.

The system responds

The altered file fails instantly even before the player spots the typo. The correct file may still be dangerous content, but it is provably the file that was expected.

Discovery

Hashes tell you whether data is the same as the data you expected. They do not judge whether the expected data was wise or safe.

Theory unlock

ONLY AFTER THE ACTION

Comparing a calculated hash with an expected hash is a standard integrity check. A match is strong evidence that the bytes have not changed since the expected hash was produced. The source of the expected hash must itself be trusted.

Debrief and reward

You caught sabotage before it reached the engine. Rocket Engine unlocked. +30 XP.

MODULE 05 | PEAK HEAT

How Blocks Are Connected

The player sees why a blockchain is more than a folder of blocks. Change an old record and every later connection becomes visibly unstable.

ROCKET PART UNLOCKED

Structural Frame - the rocket's history is now linked, checked, and difficult to rewrite.

Player leaves able to

- Explain the previous-block hash link.
- Trace the domino effect caused by changing old block data.
- Recognize that a block hash represents its header data and network rules.
- Explain why recalculating hashes locally is not enough to rewrite shared history.
- Describe 51% control as a majority-network risk, not magic invincibility.

Mission briefing

ON-SCREEN TRANSMISSION

A saboteur altered an old fuel transfer. The log still looks like a stack of containers, but its structural cables are beginning to fail. Find the first lie before the whole launch frame buckles.

Mission beat 1: Connect the history cables

PREVIOUS BLOCK HASH**Observe**

Four blocks hover in sequence. Each has its own fingerprint and a socket labeled 'previous block fingerprint'. The sockets are disconnected.

Player does

The player drags the fingerprint from each earlier block into the next block's previous-reference socket. They may try connecting a block to the wrong parent or skipping a height.

The system responds

Correct connections lock into a bright chain. Incorrect ones spark because the referenced fingerprint does not belong to the proposed parent.

Discovery

A later block carries evidence of the specific block before it. The chain is not just sorted; each link depends on the content of the previous link.

Theory unlock

ONLY AFTER THE ACTION

Before a block is sealed, its header includes the hash of the preceding block. If that earlier block changes, its hash changes. The later block's stored reference no longer matches, exposing a broken link.

Debrief and reward

You made the order of history part of the data itself. +20 XP: Link Array connected.

Mission beat 2: The domino corridor

TAMPER EVIDENCE / BLOCK HEADER

Observe

The player edits one transaction in Block 2 from 20 to 200. Block 2's seal changes. Block 3's incoming cable snaps, then Block 4's, then Block 5's.

Player does

The player attempts a repair by recalculating Block 2, then editing Block 3's previous reference and recalculating it, then repeating the process downstream. A header workbench exposes the values being sealed: height, timestamp, previous hash, transaction summary, and protocol value.

The system responds

The player can repair their local copy, but a progress meter shows the cost growing with every later block. An honest network copy remains intact beside it.

Discovery

Old fraud is not impossible to write down locally. It is expensive to hide because every later block must be rebuilt to agree with the changed past.

Theory unlock

ONLY AFTER THE ACTION

A block hash is calculated from block-header data according to the network's rules. Changing transactions changes the transaction summary or root; changing that changes the block hash; changing the block hash invalidates the next block's previous-hash reference. This is the tamper-evident domino effect.

Debrief and reward

You did not find a magical lock. You discovered a chain of costly evidence. +20 XP: Shock Braces installed.

Mission beat 3: The lone rewrite

DECENTRALIZED SECURITY / MAJORITY CONTROL

Observe

The saboteur successfully rebuilds their private altered chain. Three honest station nodes keep the original chain. A broadcast race begins.

Player does

The player allocates limited compute power between the attacker and honest validators, then watches nodes accept the best valid history under the simulation's rule set. They must explain why 'my altered copy has valid-looking hashes' is not enough.

The system responds

The attacker can verify their own chain, but cannot make independent nodes adopt it unless they can consistently outpace or out-control the honest network's ability to extend valid history. The simulation flags this as a majority-control risk, not an automatic guarantee of all outcomes.

Discovery

Hash links detect inconsistency. Network agreement decides which valid history becomes shared truth. Both are needed.

Theory unlock

ONLY AFTER THE ACTION

Blockchain security is not supplied by hashes alone. Independent nodes compare history using the protocol's consensus rules. A so-called 51% attack refers to an attacker gaining enough controlling resource in certain systems to influence which chain progresses; it does not let them create arbitrary signatures, steal any account, or erase every rule.

Debrief and reward

You separated local calculation from network acceptance. Structural Frame unlocked. +30 XP.

Decentralization and Distributed Networks

The player builds a communication network that keeps speaking when individual stations vanish. They learn that 'decentralized' and 'distributed' are shapes with tradeoffs, not interchangeable buzzwords.

ROCKET PART UNLOCKED

Communications Array - the rocket can now hear and be heard without one fragile hub.

Player leaves able to

- Distinguish centralized, decentralized, and distributed network topologies.
- Explain the roles of full, light, and validator nodes.
- Trace peer-to-peer gossip and see invalid messages stop at validation.
- Reason about crash and Byzantine fault tolerance without treating formulas as trivia.
- Design a network that survives staged outages.

Mission briefing

ON-SCREEN TRANSMISSION

The storm will knock out stations one by one. Central Command cannot be trusted as the only relay. Build a network that can keep an accurate shared signal alive.

Mission beat 1: Build the storm map

NETWORK TOPOLOGY

Observe

Six station cards are shown in three layouts: all spokes into one hub, regional hubs linked together, and a peer mesh. A solar blast strikes the central hub in each preview.

Player does

The player wires six nodes three different ways, then triggers the same outage. They choose a topology for a private company control panel, a regional payment network, and a distrustful multi-party ledger.

The system responds

The hub-and-spoke design is simple and fast but loses its decision center when the hub fails. The regional layout has several hubs. The mesh keeps alternate routes but costs more coordination.

Discovery

A network can reduce dependence on one central component in more than one way. More paths improve resilience but do not magically create agreement.

Theory unlock

ONLY AFTER THE ACTION

In a centralized topology, many clients depend on one authority. In a decentralized topology, several hubs serve parts of the network. In a distributed peer network, nodes connect across multiple paths and share work. Blockchain networks are typically peer-to-peer distributed systems, but topology alone is not consensus.

Debrief and reward

You chose a shape for a risk, not a label for a slide. +20 XP: Relay Blueprint acquired.

Mission beat 2: Pass the signal

NODES / GOSSIP PROTOCOL

Observe

A valid oxygen transaction appears at Station A. Nearby nodes blink as they receive and forward it. An invalid forged transaction arrives immediately afterward.

Player does

The player assigns each node a role: full node keeps and checks complete history, light node keeps compact evidence and asks for details when needed, validator joins the agreement process. They then choose which peers each node should forward to and place a validation gate on the forged message's route.

The system responds

The valid message spreads through multiple paths until the network has heard it. The forged message dies at the first node that rejects it; it should not be eagerly relayed as good data.

Discovery

Nodes are not decorative copies. They store, check, forward, and sometimes help decide the shared state. Gossip is many small forwards, not one central broadcast tower.

Theory unlock

ONLY AFTER THE ACTION

A node is a computer running the network software. Full nodes maintain and verify extensive ledger data; light nodes use compact proofs or headers; validators participate in the protocol that agrees on new ledger updates. In gossip networks, peers relay valid information to their neighbors, creating resilient propagation paths.

Debrief and reward

You made a signal travel because peers had reasons and paths to pass it on. +20 XP: Peer Relay live.

Mission beat 3: Hold the line

FAULT TOLERANCE / BFT INTUITION

Observe

Five stations host one node each. Outages strike Alpha, then Gamma, then Delta. Later, a seven-node network includes two nodes that send conflicting reports rather than simply going offline.

Player does

The player places backup nodes and links before the storm, then watches which outages isolate the system. In the second round they set an 'honest majority' threshold and test whether enough independent honest nodes remain to make a safe decision.

The system responds

A single central node fails on the first outage. A properly distributed setup stays active through early losses. When too few trustworthy nodes remain, the interface freezes decisions rather than guessing.

Discovery

Fault tolerance is not 'the network never fails'. It is knowing how much failure or dishonesty the design can survive before it must stop.

Theory unlock

ONLY AFTER THE ACTION

Crash fault tolerance considers nodes that fail silently. Byzantine fault tolerance also considers nodes that lie or send conflicting information. A common BFT rule of thumb is that to tolerate f malicious nodes, a system needs at least $3f + 1$ total nodes, with more than two thirds behaving honestly. The important intuition is that safety needs a strong honest supermajority.

Debrief and reward

You built for storms and liars, not just outages. Communications Array unlocked. +30 XP.

MODULE 07 | TWILIGHT

Consensus and Transaction Validation

There is no captain of Mercury's network. The player must get independent stations to settle on one safe history even when messages conflict and one participant is trying to mislead them.

ROCKET PART UNLOCKED

Navigation System - the rocket now has one shared course, not six competing coordinates.

Player leaves able to

- Explain why distributed nodes need a process for agreement.
- Experience a Byzantine-style conflicting-message problem.
- Compare the core tradeoffs of Proof of Work and Proof of Stake.
- Understand federated agreement through overlapping trust groups at a surface level.
- Use consensus ordering to resolve a double-spend conflict.

Mission briefing

ON-SCREEN TRANSMISSION

Mercury's stations have received two conflicting launch coordinates. One station is feeding different instructions to different neighbors. There is no commander to call. Form one safe decision before the rocket drifts beyond recovery.

Mission beat 1: No captain on the bridge

CONSENSUS / BYZANTINE FAULT**Observe**

Five stations must choose 'hold' or 'launch'. Four show their messages openly; a fifth sends 'launch' to some and 'hold' to others. The station displays diverge.

Player does

The player replays the message paths, marks the inconsistent sender, and asks honest stations to compare what they received. They must choose a rule that lets the honest group converge without pretending the traitor will cooperate.

The system responds

When the player trusts any single message, the network splits. When enough honest stations compare evidence and follow the same agreement rule, they settle on one safe action.

Discovery

A distributed system needs more than copies and connections. It needs a method for deciding what counts as the next shared truth when messages conflict.

Theory unlock

ONLY AFTER THE ACTION

Consensus is the process by which independent nodes agree on a valid sequence of updates. The Byzantine Generals problem is a story about reaching agreement when some participants may lie. Good consensus protocols aim for safety: honest participants should not finalize conflicting results.

Debrief and reward

You achieved agreement without discovering a hidden boss. +20 XP: Council Protocol learned.

Mission beat 2: The cost of one vote

SYBIL RESISTANCE / POW AND POS

Observe

An attacker creates 1,000 empty station badges and tries to outvote five real stations. Two defense consoles are offered: 'prove costly work' and 'lock valuable stake'.

Player does

The player gives the attacker many identity cards and observes that simple one-node-one-vote collapses. They run a small Proof of Work race, spending energy to find a valid puzzle result, then run a Proof of Stake round where validators lock collateral and risk losing it for dishonest behavior. They place tradeoff cards under each system: energy use, hardware competition, capital at risk, and penalty incentives.

The system responds

Fake names alone do not carry weight when participation requires a scarce resource. The two systems resist Sybil attacks differently and carry different costs.

Discovery

Consensus needs a way to make influence costly or accountable; otherwise a single attacker can manufacture votes.

Theory unlock

ONLY AFTER THE ACTION

Proof of Work ties block-production influence to computational work and energy. Proof of Stake ties it to value locked as collateral, with protocol penalties for certain dishonest behavior. Neither is simply 'better'; they make different security, energy, concentration, and operational tradeoffs.

Debrief and reward

You learned why 1,000 fake badges do not deserve 1,000 real votes. +20 XP: Anti-Sybil shield installed.

Mission beat 3: Settle one history

FEDERATED AGREEMENT / DOUBLE-SPEND ORDERING

Observe

Three trust circles overlap: a university relay, a payment gateway, and a civic exchange. A short optional signal says that some networks use overlapping trusted groups rather than mining or stake. Meanwhile, one station submits two 40-unit transfers from a 50-unit balance.

Player does

The player highlights the overlap that lets agreement propagate across trust circles. They then place the two conflicting transfers into an order. Validators apply the first valid transfer, update the balance, and reject the second. A final council screen shows which stations approved the safe order.

The system responds

Trust circles that do not overlap fragment into islands. With adequate overlap, local agreement can propagate. The first 40-unit transfer succeeds; the second is rejected because the remaining balance cannot fund it.

Discovery

Consensus is both agreement on validity and agreement on order. The order is what resolves two incompatible claims on the same funds.

Theory unlock

ONLY AFTER THE ACTION

Some federated Byzantine agreement designs let each node choose trust sets, sometimes called quorum slices. If those local trust groups overlap appropriately, agreement can spread across the network. This is a surface-level preview of the family of ideas used by Stellar's consensus design; Mercury does not require protocol implementation details. Across consensus systems, ordering turns a double-spend race into one accepted transaction and one rejected conflict.

Debrief and reward

You found the shared path and settled the disputed fuel. Navigation System unlocked. +30 XP.

Immutability, Transparency, and Choosing Blockchain

Before launch, the player must prove that they understand the cost of the machine they repaired. Blockchain's strengths matter only when they are worth its tradeoffs.

ROCKET PART UNLOCKED

Heat Shield - a final judgment tool that protects CosmosX from blockchain-for-everything thinking.

Player leaves able to

- Explain immutability as practical resistance to changing shared history, not absolute magic.
- Use transparency and traceability to audit an asset's provenance.
- Name key blockchain tradeoffs: speed, storage, fees, privacy, and responsibility.
- Recognize real use cases such as multi-party settlement, provenance, credentials, tokenization, and open finance.
- Choose database, shared database, blockchain, or no persistent system from the actual trust problem.

Mission briefing

ON-SCREEN TRANSMISSION

The rocket is built. The final risk is bad judgment: using a powerful system where a simple one would do, or using a simple system where no party can safely own the truth. Choose the right architecture before launch.

Mission beat 1: Trace the rare mineral

IMMUTABILITY / TRANSPARENCY / TRACEABILITY

Observe

A rare dilithium canister passes through a miner, refiner, shipper, and launch crew. One crew member claims it was replaced. The player sees a public chain of custody but no exposed private messages.

Player does

The player follows the canister backward through dated custody events, opens a proof that links it to its origin, and flags the one handoff that does not match its recorded evidence. They try to erase an old handoff and see later links become disputed and the network reject the local rewrite.

The system responds

The player can trace provenance without calling each organization one by one. The record is visible for audit, but sensitive details can still be represented selectively or kept off-chain depending on system design.

Discovery

Immutability is the practical difficulty of changing accepted history. Transparency makes shared records inspectable. Traceability lets people follow an asset or decision through time.

Theory unlock**ONLY AFTER THE ACTION**

Blockchain history is often described as immutable because changing an accepted old record requires overcoming hash links, consensus, and independent copies. It is more accurate to say tamper-evident and very difficult to alter under the network's security assumptions. Public visibility can improve auditability, but it creates privacy design obligations too.

Debrief and reward

You did not trust a claim about provenance; you followed the evidence. +20 XP: Audit Shield fitted.

Mission beat 2: Pay the real price**TRADEOFFS / LIMITATIONS****Observe**

Mission Control offers two systems for a high-frequency cafeteria checkout: a central database with instant writes, or a replicated blockchain ledger with fees and slower agreement. A second screen shows a lost private key locking a station out of its own funds.

Player does

The player adjusts transaction volume, block capacity, node storage, and fee budgets. They must keep a service operating while paying for its security model. They respond to a key-loss incident by distinguishing 'the network is broken' from 'the user lost their authorization'.

The system responds

The normal database wins the cafeteria scenario on cost and speed. The blockchain's replicated history consumes more storage and coordination. No support desk can honestly recreate a lost private key in a self-custody design.

Discovery

Decentralized verification has costs. It can improve resilience and independent auditability while reducing speed, simplicity, privacy, or recoverability.

Theory unlock**ONLY AFTER THE ACTION**

Blockchain tradeoffs include throughput limits, growing replicated storage, transaction fees, latency while waiting for agreement, and user responsibility for keys. These are not footnotes; they are part of the architecture decision. A good designer states the tradeoff before choosing the technology.

Debrief and reward

You chose honesty about cost over hype. +20 XP: Heat Shield reinforced.

Mission beat 3: The launch architect

USE CASES / FINAL DECISION MATRIX

Observe

Eight live project requests arrive: employee lunch orders, rival companies settling shared shipping charges, a university issuing verifiable diplomas, a family photo album, a tokenized solar farm, a public charity donation trail, an internal game leaderboard, and a cross-border payout network.

Player does

For each request, the player asks three visible questions: Are there multiple writers? Do they distrust one another? Is a trusted operator acceptable? They drag the project to 'database', 'shared database/API', 'blockchain', or 'no durable system'. A consequence card explains the decision in ordinary language.

The system responds

The player sees several valid answers with context. A university may issue credentials through a centralized registrar and still publish verifiable proofs; a blockchain is justified only where independent verification or multi-party control adds real value. The rival settlement, provenance, tokenization, and some cross-border payment cases make the strongest blockchain candidates.

Discovery

Blockchain is a coordination tool, not a moral upgrade. The question is whether its extra trust model is necessary.

Theory unlock

ONLY AFTER THE ACTION

Use blockchain when multiple parties must write to or verify important shared state, they do not fully trust each other, and a central operator is unacceptable or insufficient. Useful applications include multi-party settlement, provenance, digital credentials, tokenization, and open financial protocols. If one accountable organization can safely run the service, use a database.

Debrief and reward

You are ready to choose blockchain deliberately, including the times you should reject it. Heat Shield unlocked. +40 XP.

FINALE

Mercury Rescue: Launch Sequence

The finale is not a separate exam. It is one connected rescue sequence that reuses the mechanics the player has already mastered. No new vocabulary is introduced here.

FINAL TRANSMISSION

Mercury is ready to launch, but the solar storm is back. Repair the record system, move the oxygen, restore the chain, reconnect peers, settle one safe history, and choose the technology that can hold the launch registry after you leave.

1. Central blackout

Keep the evacuation moving while the central command screen is unavailable. The player identifies the missing source of truth.

2. Oxygen queue

Validate three emergency transfers and reject one forged or conflicting request.

3. Cargo seal

Pack valid transfers into a block and verify the header information.

4. Tamper scan

Find a changed launch transmission by comparing its fingerprint with the expected seal.

5. Broken history

Locate the earliest altered block and reconnect the downstream references.

6. Storm mesh

Bring enough independent peer nodes online to keep the signal flowing after an outage.

7. One shared course

Expose a conflicting message and order two competing payments so the network finalizes one safe result.

8. Architect's choice

Select a normal database, shared database, blockchain, or no durable system for the post-launch registry, then explain the trust reason.

Completion reflection

PLAYER-FACING REFLECTION

You did not learn that blockchain is a magic database. You learned why people built it: sometimes strangers need one shared history, and no single owner is trusted to write it alone. You also learned the responsibility that comes with that choice.

Rewards: Mercury mission patch, complete Launch Rocket, planet progress at 100%, and access to Venus: Wallets and Cryptography.

Learning-objective coverage check

This redesign retains every major objective in the current Mercury curriculum while reducing the number of isolated answer-entry tasks. Related concepts are experienced together in meaningful system repairs.

Why blockchain

Centralization, trust, single failure, distributed records, appropriate technology choice

Transactions

Payloads, signed requests, append-only ledger, lifecycle, mempool, validation, double spend

Blocks

Headers, bodies, genesis, capacity, throughput, chronological linkage

Hashing

Determinism, fixed length, one-way intuition, avalanche effect, integrity, encryption distinction

Linked history

Previous hashes, domino effect, header sealing, consensus plus majority-control risk

Networks

Topology, node roles, gossip, crash faults, Byzantine faults, $3f + 1$ intuition

Consensus

Byzantine agreement, Sybil resistance, PoW, PoS, federated overlap preview, ordering

Judgment

Immutability, transparency, traceability, limitations, applications, decision matrix

Writing and interaction checklist

- Start every screen with an event, object, or choice - never a definition paragraph.
- Use one concept name only after the player has seen its consequence.
- Keep revealed theory to 80 to 130 words per beat and use one familiar analogy where it helps.
- Make every wrong choice legible: show what changed, why it matters, and how to try again.
- Do not reward guessing. Reward an action that protects, repairs, verifies, or explains the system.
- Keep a visible rocket and planet-progress payoff after every module.
- Use a replay twist rather than a multiple-choice quiz whenever possible.